Relational database model

Explanation

A) Users ↔ Profiles (1-to-1)

- **users** stores authentication/security: email, password hash, role, status.
- **profiles** stores public-facing info: display name, bio, avatar, etc.
- Relationship: **one user has exactly one profile**.
  - Enforced by `profiles.user_id UNIQUE` + `FOREIGN KEY`.

B) Directory tags (many-to-many)

To filter/search by skills/interests:

- **tags** stores unique tag names ("Java", "DevSecOps", "Kiné", etc.).
- **profile_tags** links profiles to tags.
- Relationship: **one profile can have many tags** and **one tag can belong to many profiles**.

C) Events ↔ Users (1-to-many) + RSVP (many-to-many)

- **events**: created/owned by one organizer user (`organizer_user_id`).
- **event_rsvps**: links users to events with a status (`going`, `interested`, etc.).
- Relationship:
  - **one user can organize many events** (1-to-many)
  - **users attend many events and events have many users** (many-to-many via `event_rsvps`)
  - Enforced by unique `(event_id, user_id)` so a user can RSVP only once per event.

D) Email outbox (queue table)

- **email_outbox** stores emails to send asynchronously (queued/sent/failed).
- This prevents slow email sending from blocking API requests.

E) Subscription

Subscribe

- Input: email
- Behavior:
  - Normalize email (trim, lowercase)
  - Insert it if it doesn't exist
  - If it already exists, do nothing (idempotent)

Unsubscribe

- Input: email (or better: an unsubscribe token in the email link)
- Behavior:

- Remove the email from the table (or mark as unsubscribed — but you asked to remove status, so we delete)

Send emails

- When you send a newsletter or event announcement:
  - Query all emails in `subscriptions`
  - Send to all

That's it.

CODE

ENUM

```
DO $$ BEGIN
  CREATE TYPE user_role AS ENUM ('member', 'admin');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE user_status AS ENUM ('active', 'suspended', 'deleted');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE profile_visibility AS ENUM ('public', 'members_only', 'hidden');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE event_visibility AS ENUM ('public', 'members_only', 'private_invite');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE event_status AS ENUM ('scheduled', 'cancelled', 'completed');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE rsvp_status AS ENUM ('going', 'interested', 'not_going', 'waitlist');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE email_status AS ENUM ('queued', 'sent', 'failed');
```

```
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;
```

USERS TABLE

```
CREATE TABLE users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email             TEXT NOT NULL UNIQUE,
  password_hash     TEXT NOT NULL,
  role              user_role NOT NULL DEFAULT 'member',
  status            user_status NOT NULL DEFAULT 'active',
  email_verified_at TIMESTAMPTZ NULL,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

PROFILES TABLE (THIS IS THE DIRECTORY)x

```
CREATE TABLE profiles (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL UNIQUE,

  name          TEXT NOT NULL,
  surname       TEXT NOT NULL,
  city          TEXT NULL,
  country       TEXT NULL,
  bana_quarter  TEXT NUL
  occupation    TEXT NULL

  visibility    profile_visibility NOT NULL DEFAULT 'members_only',
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_profiles_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE CASCADE
);

CREATE INDEX idx_profiles_visibility ON profiles(visibility);
CREATE INDEX idx_profiles_city ON profiles(city);
CREATE INDEX idx_profiles_country ON profiles(country);
```

EVENTS TABLE

```
CREATE TABLE events (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title             TEXT NOT NULL,
```

```
  description       TEXT NULL,
  start_at          TIMESTAMPTZ NOT NULL,
  end_at            TIMESTAMPTZ NULL,
  location_text     TEXT NULL,
  capacity          INTEGER NULL CHECK (capacity IS NULL OR capacity >= 1),

  visibility        event_visibility NOT NULL DEFAULT 'members_only',
  status            event_status NOT NULL DEFAULT 'scheduled',
  organizer_user_id UUID NOT NULL,

  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_events_organizer
    FOREIGN KEY (organizer_user_id)
    REFERENCES users(id)
    ON DELETE RESTRICT
);
```

EVENT RSVPS

```
CREATE TABLE event_rsvps (
  event_id   UUID NOT NULL,
  user_id    UUID NOT NULL,
  status     rsvp_status NOT NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

  PRIMARY KEY (event_id, user_id),

  FOREIGN KEY (event_id) REFERENCES events(id) ON DELETE CASCADE,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

SUBSCRIPTIONS

```
CREATE TABLE subscriptions (
  email       TEXT PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The Whole Code

```
-- ==========================================
-- 0) Extensions (needed for gen_random_uuid)
-- ==========================================
```

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- ==========================================
-- 1) Enum types
-- ==========================================
DO $$ BEGIN
  CREATE TYPE user_role AS ENUM ('member', 'admin');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE user_status AS ENUM ('active', 'suspended', 'deleted');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE profile_visibility AS ENUM ('public', 'members_only', 'hidden');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE event_visibility AS ENUM ('public', 'members_only', 'private_invite');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE event_status AS ENUM ('scheduled', 'cancelled', 'completed');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE rsvp_status AS ENUM ('going', 'interested', 'not_going', 'waitlist');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TYPE email_status AS ENUM ('queued', 'sent', 'failed');
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

-- ==========================================
-- 2) USERS (Auth/Security)
-- ==========================================
CREATE TABLE IF NOT EXISTS users (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email             TEXT NOT NULL UNIQUE,
  password_hash     TEXT NOT NULL,
  role              user_role NOT NULL DEFAULT 'member',
  status            user_status NOT NULL DEFAULT 'active',
  email_verified_at TIMESTAMPTZ NULL,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ==========================================
-- 3) PROFILES (Directory)
--   - 1:1 with users
--   - This table is what your directory lists
-- ==========================================
CREATE TABLE IF NOT EXISTS profiles (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL UNIQUE,

  name          TEXT NOT NULL,
  surname       TEXT NOT NULL,
  occupation    TEXT NULL,
  city          TEXT NULL,
  country       TEXT NULL,
  bana_quarter  TEXT NUL

  visibility    profile_visibility NOT NULL DEFAULT 'members_only',

  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_profiles_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_profiles_visibility ON profiles(visibility);
CREATE INDEX IF NOT EXISTS idx_profiles_city ON profiles(city);
CREATE INDEX IF NOT EXISTS idx_profiles_country ON profiles(country);
CREATE INDEX IF NOT EXISTS idx_profiles_surname ON profiles(surname);

-- ==========================================
-- 4) EVENTS
--   - One organizer user creates many events
-- ==========================================
CREATE TABLE IF NOT EXISTS events (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title             TEXT NOT NULL,
  description       TEXT NULL,
  start_at          TIMESTAMPTZ NOT NULL,
  end_at            TIMESTAMPTZ NULL,

  location_text     TEXT NULL,
  capacity          INTEGER NULL CHECK (capacity IS NULL OR capacity >= 1),

  visibility        event_visibility NOT NULL DEFAULT 'members_only',
  status            event_status NOT NULL DEFAULT 'scheduled',
```

```
  organizer_user_id UUID NOT NULL,

  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT now(),

  CONSTRAINT fk_events_organizer
    FOREIGN KEY (organizer_user_id)
    REFERENCES users(id)
    ON DELETE RESTRICT
);

CREATE INDEX IF NOT EXISTS idx_events_start_at ON events(start_at);
CREATE INDEX IF NOT EXISTS idx_events_visibility ON events(visibility);
CREATE INDEX IF NOT EXISTS idx_events_status ON events(status);

-- ==========================================
-- 5) EVENT RSVPs (many-to-many via join table)
--   - One user can RSVP once per event
-- ==========================================
CREATE TABLE IF NOT EXISTS event_rsvps (
  event_id   UUID NOT NULL,
  user_id    UUID NOT NULL,
  status     rsvp_status NOT NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),

  PRIMARY KEY (event_id, user_id),

  CONSTRAINT fk_event_rsvps_event
    FOREIGN KEY (event_id) REFERENCES events(id)
    ON DELETE CASCADE,

  CONSTRAINT fk_event_rsvps_user
    FOREIGN KEY (user_id) REFERENCES users(id)
    ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_event_rsvps_user_id ON event_rsvps(user_id);
CREATE INDEX IF NOT EXISTS idx_event_rsvps_status ON event_rsvps(status);

-- ==========================================
-- 6) SUBSCRIPTIONS (email-only, subscribed to everything)
--   - No user_id, no status, no topics
-- ==========================================
CREATE TABLE IF NOT EXISTS subscriptions (
  email       TEXT PRIMARY KEY,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
-- ==========================================
-- 7) EMAIL OUTBOX (queue for reliable sending)
-- ==========================================
CREATE TABLE IF NOT EXISTS email_outbox (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  to_email     TEXT NOT NULL,
  template_key TEXT NOT NULL,
  payload      JSONB NOT NULL DEFAULT '{}'::jsonb,
  status       email_status NOT NULL DEFAULT 'queued',
  last_error   TEXT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  sent_at      TIMESTAMPTZ NULL
);

CREATE INDEX IF NOT EXISTS idx_email_outbox_status_created_at
  ON email_outbox(status, created_at);

-- ==========================================
-- 8) updated_at trigger helper (optional)
-- ==========================================
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DO $$ BEGIN
  CREATE TRIGGER trg_users_updated_at
  BEFORE UPDATE ON users
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TRIGGER trg_profiles_updated_at
  BEFORE UPDATE ON profiles
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TRIGGER trg_events_updated_at
  BEFORE UPDATE ON events
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;

DO $$ BEGIN
  CREATE TRIGGER trg_event_rsvps_updated_at
```

```
  BEFORE UPDATE ON event_rsvps
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
EXCEPTION WHEN duplicate_object THEN NULL;
END $$;
```